

Тема 13 – Нелинейни АТД

1. Дървовидни структури.
2. Двоични дървета.
3. Подредени дървета.
4. Балансирани (AVL) дървета.

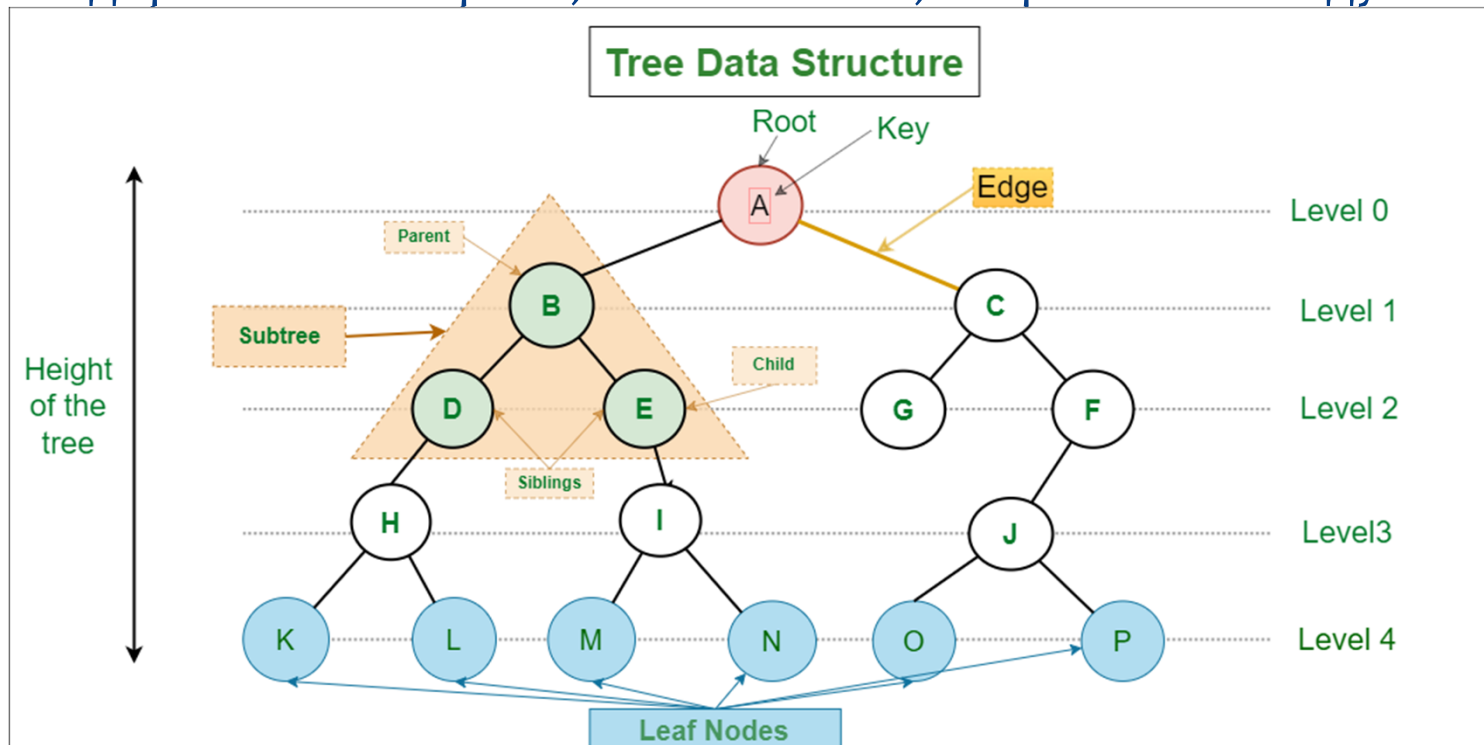
Тема 13 – Нелинейни АТД

Дървовидната структура на данни е йерархична структура, която се използва за представяне и организиране на данни по начин, който е лесен за навигиране и търсене. Това е колекция от възли, които са свързани с ръбове и има йерархична връзка между възлите.

Най-горният възел на дървото се нарича корен, а възлите под него се наричат дъщерни възли. Всеки възел може да има множество дъщерни възли и тези дъщерни възли също могат да имат свои собствени дъщерни възли, образувайки рекурсивна структура.

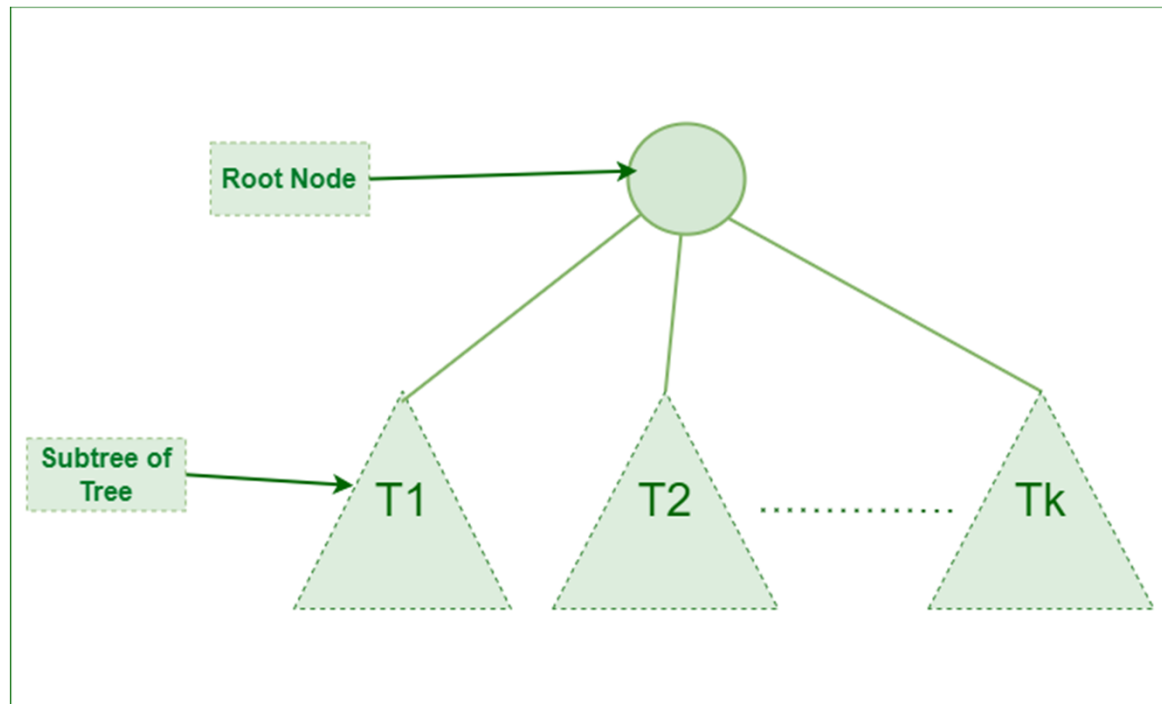
Тема 13 – Нелинейни АТД

Дървовидната структура от данни е специализиран метод за организиране и съхраняване на данни в компютъра, за да се използват по-ефективно. Състои се от централен възел, структурни възли и подвъзли, които са свързани чрез ръбове. Можем също да кажем, че структурата на данните на дървото има корени, клони и листа, свързани помежду си.



Тема 13 – Нелинейни АТД

Рекурсивната дефиниция на дървовидната структура на данни е: **Едно дърво се състои от корен и нула или повече поддървета $T_1, T_2, \dots, T_k$, така че да има ребро от корена на дървото до корена на всяко поддърво.**



Тема 13 – Нелинейни АТД

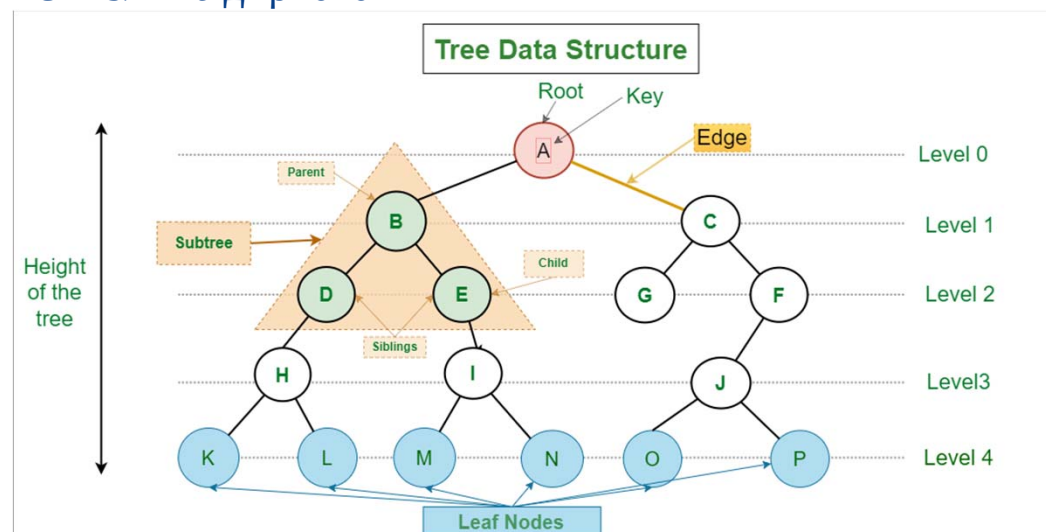
Основни терминологии в дървовидната структура на данните:

Родителски възел: Възелът, който е предшественик на възел, се нарича родителски възел на този възел. {B} е родителският възел на {D, E}.

Дъщерен възел: Възелът, който е непосредствен наследник на възел, се нарича дъщерен възел на този възел. Примери: {D, E} са дъщерните възли на {B}.

Основен възел (корен): Най-горният възел на дървото или възелът, който няма родителски възел, се нарича основен възел (корен). {A} е основният възел на дървото. Едно непразно дърво трябва да съдържа точно един коренов възел и точно един път от корена до всички останали възли на дървото.

Листен възел или външен възел: Възлите, които нямат дъщерни възли, се наричат листови възли. {K, L, M, N, O, P} са листните възли на дървото.



Тема 13 – Нелинейни АТД

Предшественик на възел: Всички предшестващи възли по пътя на корена към този възел се наричат предшественици на този възел. $\{A, B\}$ са предшестващите възли на възела $\{E\}$

Потомък: Всеки възел наследник по пътя от листовия възел до този възел. $\{E, I\}$ са потомците на възела $\{B\}$.

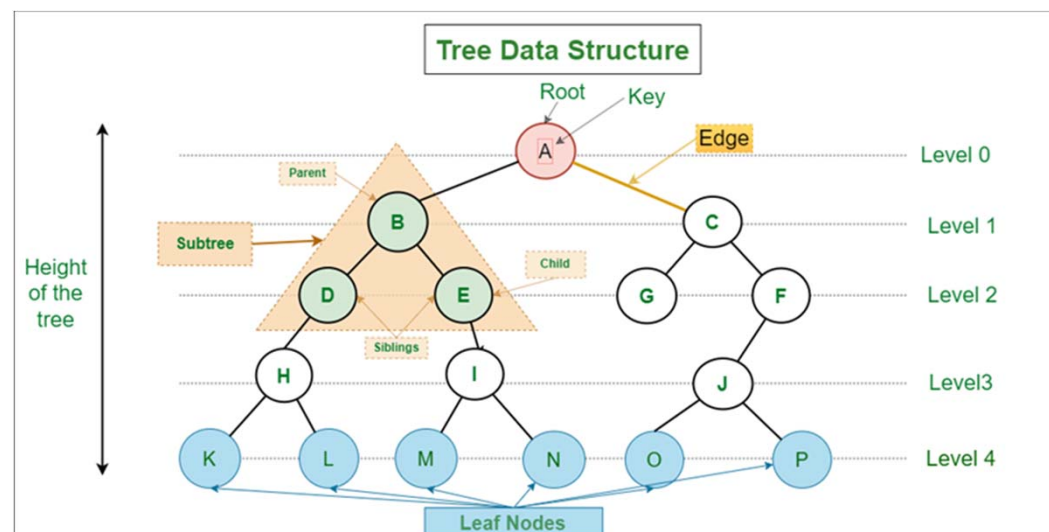
Братя и сестри: Децата на един и същи родителски възел се наричат братя и сестри. $\{D, E\}$ се наричат братя и сестри.

Ниво на възел: Броят на ръбовете по пътя от основния възел до този възел. Основният възел има ниво 0.

Вътрешен възел: Възел с поне един дъщерен възел се нарича вътрешен възел.

Съсед на възел: Родителските или дъщерните възли на този възел се наричат съседи на този възел.

Поддърво: Всеки възел на дървото заедно с неговия наследник.



Тема 13 – Нелинейни АТД

Свойства на дърво:

Брой ръбове: Ръбът може да се дефинира като връзката между два възела. Ако едно дърво има N възли, то ще има $(N-1)$ ръба. Има само един път от всеки възел до всеки друг възел на дървото.

Дълбочина на възел: Дълбочината на възел се определя като дължината на пътя от корена до този възел. Всеки ръб добавя 1 единица дължина към пътя. Така че може да се дефинира и като броя на ръбовете в пътя от корена на дървото до възела.

Височина на възел: Височината на възел може да се определи като дължината на най-дългия път от възела до листовия възел на дървото.

Височина на дървото: Височината на дървото е дължината на най-дългия път от корена на дървото до листен възел на дървото.

Степен на възел: Общият брой поддървета, прикрепени към този възел, се нарича степен на възела. Степента на листовия възел трябва да бъде 0. Степента на дървото е максималната степен на възел сред всички възли в дървото.

Тема 13 – Нелинейни АТД

Защо да използваме дървовидна структура на данните?

1. Една от причините да използвате дървета може да е, защото искате да съхранявате информация, която естествено образува йерархия. Например файловата система на компютър
2. Дърветата (с известно подреждане, напр. BST) осигуряват умерено бърз достъп/търсене (по-бързо от свързания списък и по-бавно от масивите).
3. Подобно на свързаните списъци и за разлика от масивите, дърветата нямат горна граница за броя на възлите, тъй като възлите са свързани с помощта на указатели.

Тема 13 – Нелинейни АТД

Синтаксис за деклариране:

```
struct Node  
{  
    int data;  
    struct Node *left_child;  
    struct Node *right_child;  
}
```

Тема 13 – Нелинейни АТД

Основни операции за дървото:

1. Инициализиране на дърво
2. Добавяне на елемент в дърво
3. Обхождане на дърво
4. Търсене на елемент.
5. Премахване на елемент от дърво

Тема 13 – Нелинейни АТД

Инициализация на двоично дърво. На указателя към корена на дървото се присвоява стойност NULL при дефинирането на структурата.

Пример: Инициализация на двоично дърво

struct elem

```
{    char key;  
    elem *left, *right;  
} *root=NULL;
```

Тема 13 – Нелинейни АТД

Обхождане на двоично дърво (посещаване на всеки възел по веднъж). Тъй като дървото е нелинейна структура, обхождането му може да става по различни начини. Прилагат се методи за обхождане в ширина и в дълбочина, като последния се извършва по три стандартни начина:

в прав ред – PREORDER

във вътрешен ред (или симетрично обхождане) - INORDER

в обратен ред – POSTORDER

Както се вижда от дефиницията, дървото е рекурсивна структура данни. Затова най-удобно основните операции да се реализират рекурсивно, макар че не се изключват възможности и за итеративна реализация.

Тема 13 – Нелинейни АТД

Обхождане на двоично дърво в прав ред (Корен-Ляво-Дясно или Preorder)

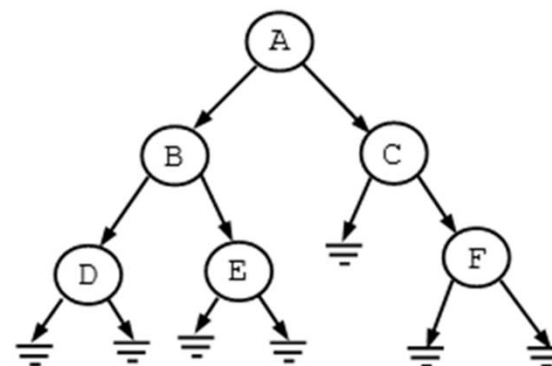
```
void preorder(elem *t)
{
    if (t)
    {
        cout<<t->key<<" ";
        preorder(t->left);
        preorder(t->right);
    }
}
```

Ниво 0

Ниво 1

Ниво 2

Ниво 3



При обхождането в прав ред се възлите се посещават в следната последователност: корен - ляво поддърво - дясно поддърво, което за горния пример на двоично дърво ще даде такава последователност: ABDEC F.

Тема 13 – Нелинейни АТД

Обхождане на двоично дърво във вътрешен ред (Ляво-Корен-Дясно или Inorder)

```
void inorder (elem *t)
```

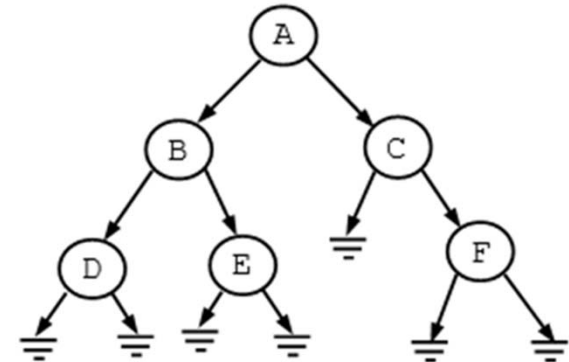
```
{  
  if (t)  
  {  
    inorder (t->left);  
    cout<<t->key<<" ";  
    inorder (t->right);  
  }  
}
```

Ниво 0

Ниво 1

Ниво 2

Ниво 3



Обхождането във вътрешен ред се извършва така: ляво поддърво - корен - дясно поддърво, т.е. за горния пример – DBEACF.

Тема 13 – Нелинейни АТД

Обхождане на двоично дърво в обратен ред (Ляво-Дясно-Корен или **Postorder**)

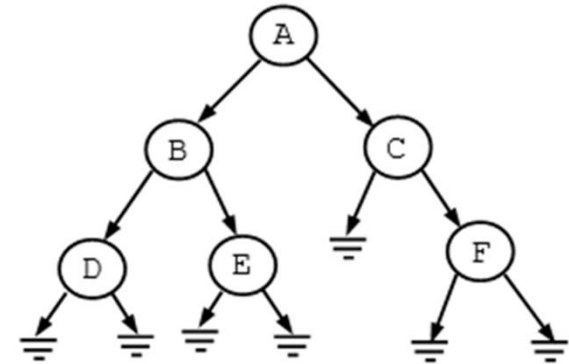
```
void postorder(elem *t)
{
    if (t)
    {
        postorder(t->left);
        postorder(t->right);
        cout<<t->key<<" ";
    }
}
```

Ниво 0

Ниво 1

Ниво 2

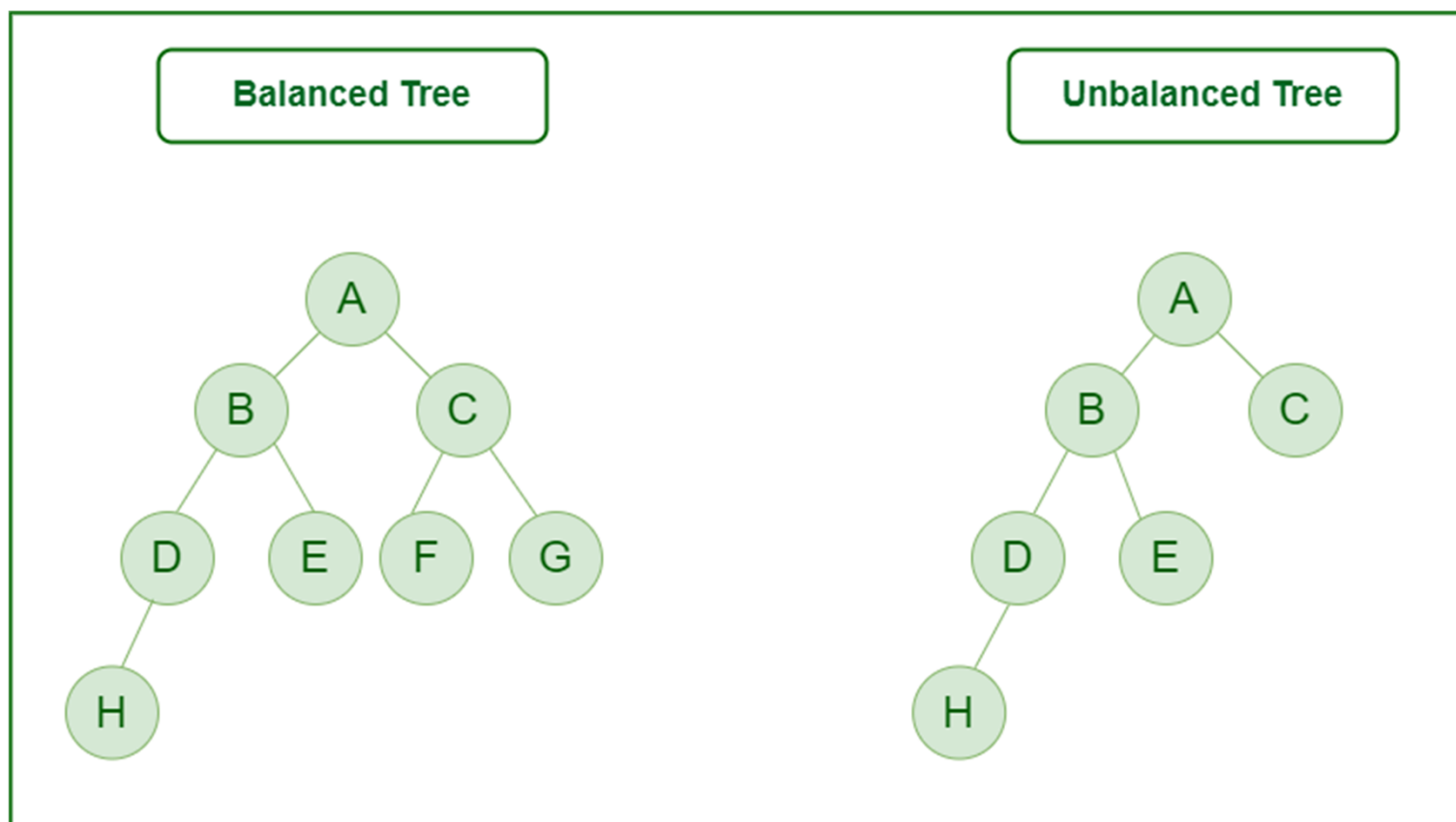
Ниво 3



Обхождането в обратен ред означава следната последователност на посещаваните възли: ляво поддърво - дясно поддърво - корен, за вече споменатия пример тя е **DEBFCA**.

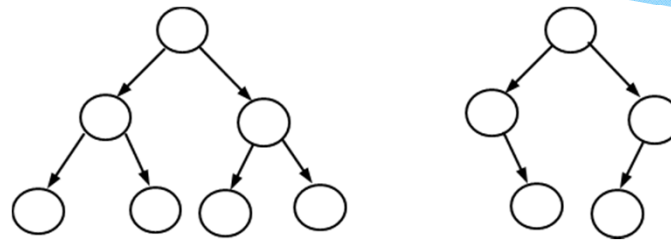
Тема 13 – Нелинейни АТД

Всяко двоично дърво може да бъде балансирано. Двоично дърво се нарича балансирано или балансирано по височина, ако разликата във височините на поддърветата на всеки негов възел е най-много 1:



Тема 13 – Нелинейни АТД

Едно двоично дърво се нарича идеално балансирано или с балансирано тегло, ако всеки негов възел има ляво и дясно поддърво, в които броят на възлите се различава най-много с 1:



Очевидно е, че всяко идеално балансирано дърво (с балансирано тегло) също така е дърво с балансирана височина. Обратното твърдение не е вярно.

Идеално балансираното дърво може да бъде пълно, ако за всеки възел разликата в броя на възлите в неговите поддървета е нула. Лявото дърво на горната фигура е пълно.

За да бъде двоичното дървото пълно, броят на възлите му трябва да е нечетен.

Тема 13 – Нелинейни АТД

Балансът е важен фактор при работа с двоични дървета за търсене, където от значение е времето за търсене и не трябва да има отделни дълги клонове.

Балансираните дървета често наричат AVL-дървета в чест на Adelson-Velski и Landis - двамата учени, създали най-много алгоритми за този клас дървовидни структури.

Основните операции с балансираните дървета - добавяне, премахване и търсене на елемент с зададена ключова стойност - имат сложност **$O(\log_2 N)$** .

Ако **$h(N)$** е височината на балансирано двоично дърво с **N** вътрешни възела, то

$$\log_2(N+1) \leq h(N) \leq 1.4404 \log_2(N+2) - 0.328$$

Във всеки възел на такава структура необходимо е да се съхранява информация за баланса на съответните поддървета. Динамичната променлива ***bal***, участваща в описанието на структурата на балансирано двоично дърво, е дадена в следващия фрагмент:

Тема 13 – Нелинейни АТД

Дефиниция на балансирано двоично дърво

```
struct tree{  
    int key;  
    int bal;  
    //bal=0 - двата клона, излизащи от възела са с еднаква дължина  
    //bal=-1 - левия клон е по-дълъг  
    //bal=+1 - десния е по-дълъг  
    tree *left,*right;  
};
```

Тема 13 – Нелинейни АТД

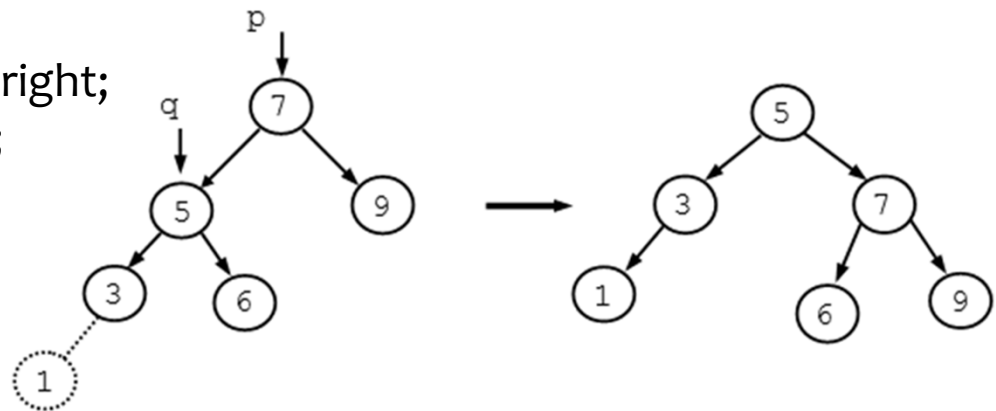
Всяко двоично дърво може да бъде балансирано. Необходими са четири основни операции за балансиране на двоично дърво, наречени ротации:

- **Дясна ротация.** Тази операция е необходима тогава, когато във вече балансираното двоично дърво се добавя елемент към най-левия клон:

Функция за дясна ротация на двоично дърво

```
void R_rot (tree * &p)
{tree *q;
 if(p->bal==-1){
```

```
    q=p->left;
    if(q->bal==-1) {
        p->left=q->right;
        q->right=p;
        p->bal=0;
        p=q;}}}
```



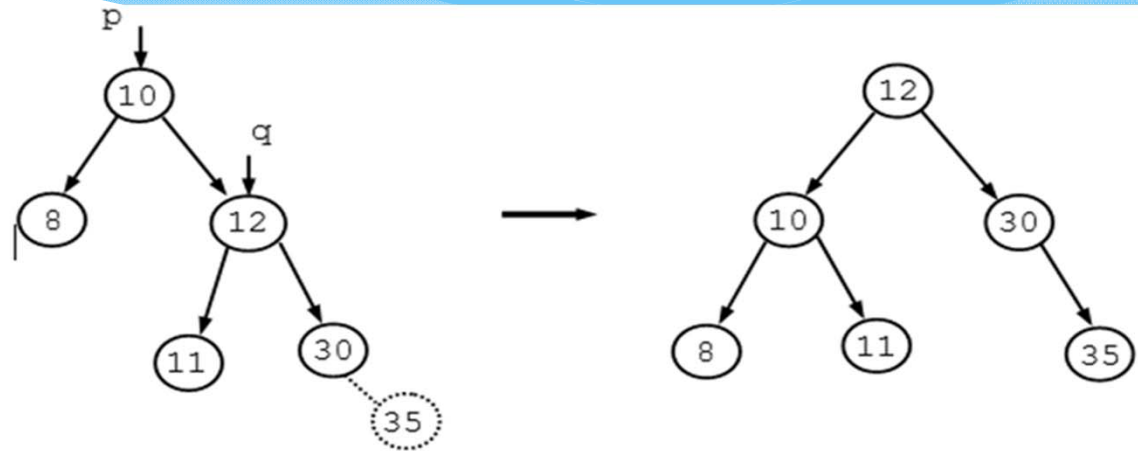
Тема 13 – Нелинейни АТД

Лява ротация. Тази операция е необходима тогава, когато във вече балансираното двоично дърво се добавя елемент към най-деснияя клон:

Функция за лява ротация на двоично дърво

```
void L_rot(tree * &p)
{tree *q;
 if(p->bal==1){
```

```
    q=p->right;
    if(q->bal==1){
        p->right=q->left;
        q->left=p;
        p->bal=0;
        p=q;
    }
}}
```



Тема 13 – Нелинейни АТД

Ляво-дясна ротация. Тази операция е необходима тогава, когато във вече балансираното двоично дърво се добавя елемент към дясното листо на по-високото ляво поддърво, в случая 21:

Функция за ляво-дясна ротация на двоично дърво

```
void L R rot(tree * &p)
```

```
{tree *q,*s;
```

```
if(p->bal==-1) {
```

```
q=p->left;
```

```
if(q->bal==1) {
```

```
s=q->right;
```

```
q->right=s->left;
```

```
s->left=q;
```

```
p->left=s->right;
```

```
s->right=p;
```

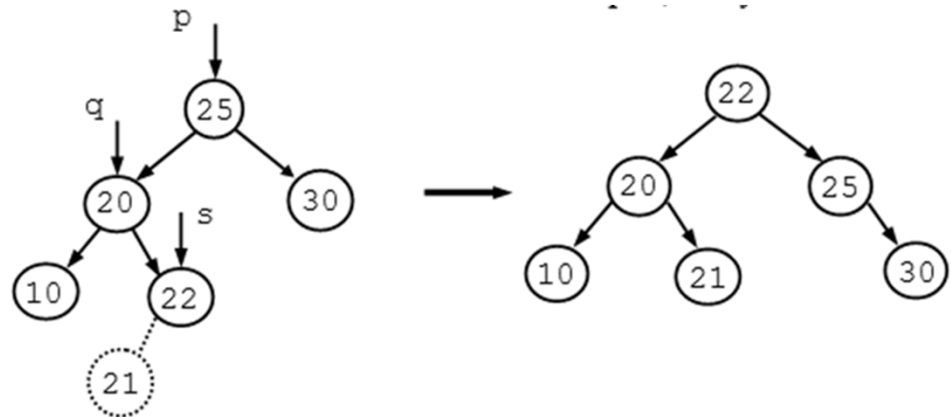
```
if(s->bal==-1) p->bal=1;
```

```
else p->bal=0;
```

```
if(s->bal==1) q->bal=-1;
```

```
else q->bal=0;
```

```
p=s; } }
```

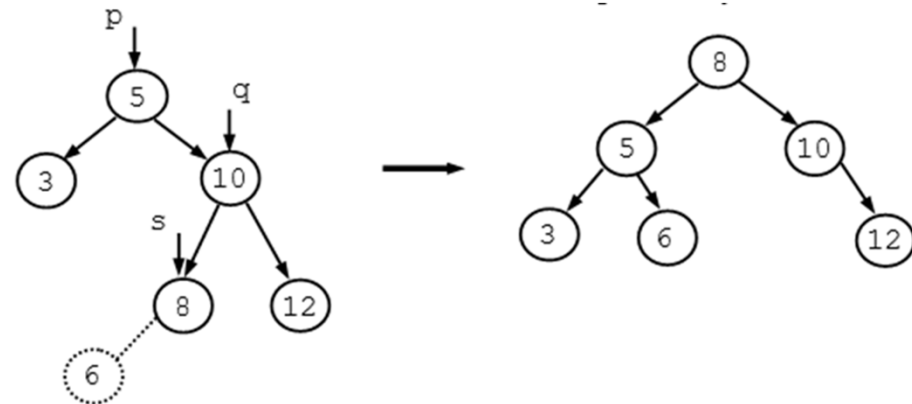


Тема 13 – Нелинейни АТД

Дясно-лява ротация. Тази операция е необходима тогава, когато във вече балансираното двоично дърво се добавя елемент към лявото листо на по-високото дясно поддърво, в случая 6:

Функция за дясно-лява ротация на двоично дърво

```
void R_L_rot(tree * &p)
{ tree *q,*s;
  if(p->bal==1){
    q=p->right;
    if(q->bal==-1) {
      s=q->left;
      q->left=s->right;
      s->right=q;
      p->right=s->left;
      s->left=p;
      if(s->bal==1) p->bal=1;
      else p->bal=0;
      if(s->bal==-1) q->bal=1;
      else q->bal=0;
      p=s;    }    } }
```



Тема 14 – Мрежови структури

1. АТД-граф. Основни дефиниции и алгоритми за работа. Алгоритми DSF и BSF. Алгоритми на Dijkstra, Ford-Belman, Floyd за търсене на минимални пътища. Суперпозиция. Транзитивно затваряне на ориентиран граф. Топологично сортиране в ацикличен граф.

Тема 14 – Мрежови структури

ГРАФ

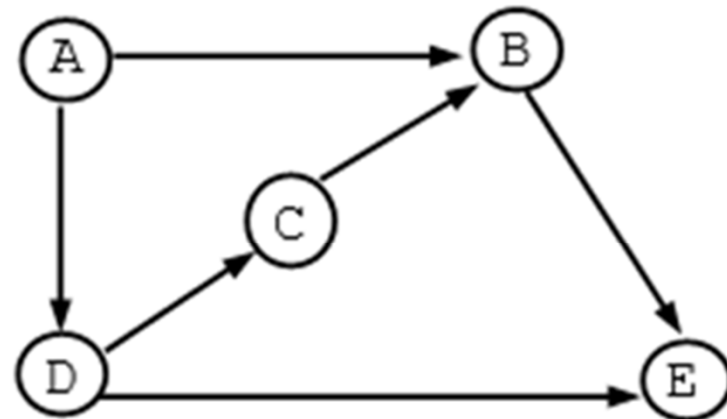
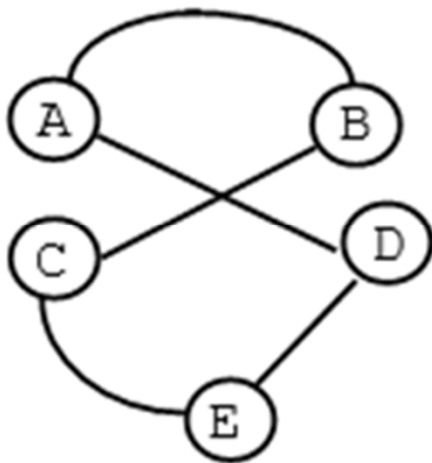
Структурата граф подобно на дървото се състои от непразно множество върхове (**възли**) V и множество дъги (**ребра**) E . Графът е подмножество от декартовото произведение на двете множества - V и E :

$$G = \{V, E\}$$

За разлика от дървото един в граф е възможно да има обратни връзки. Всяка дъга еднозначно се дефинира чрез двойката възли, които съединява, например (A,F) или (A,B) .

Тема 14 – Мрежови структури

Ако двойката е подредена, т.е. дъгата има посока, графът се нарича насочен или ориентиран, в противен случай графът е неориентиран.



Тема 14 – Мрежови структури

Една дъга X , която се описва като $X=(A,B)$, се нарича **инцидентна** с върховете A и B , т.е. върховете, които съединява или от които влиза и излиза.

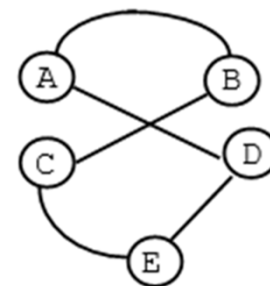
Върхове, свързани с една дъга, се наричат **съседни**.

Две и повече дъги, излизащи от един връх, се наричат **инцидентни** (например (A,B) и (A,D)).

Броят на ребрата, инцидентни с даден връх V_i , се нарича **степен на върха**. Означава се с $d(V_i)$.

Например $d(A)=2$.

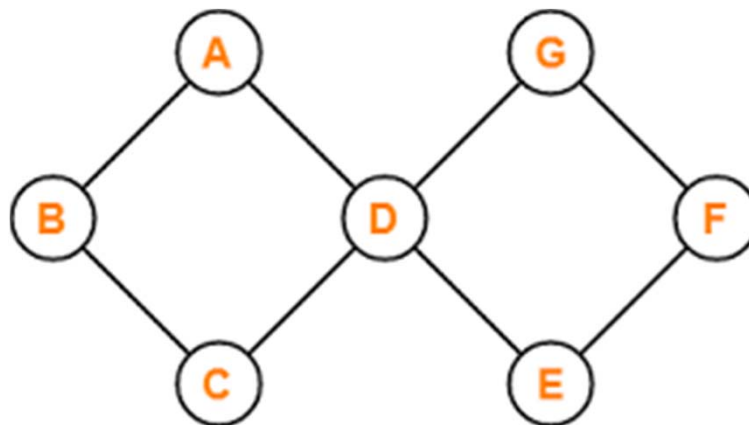
Ако $d(V_i)=0$, върхът V_i се нарича **изолиран**.



Тема 14 – Мрежови структури

Път между върховете V_i и V_j е последователността от дъгите, които ги свързват. **Дължината на пътя е броят на тези дъги.** Пътят се нарича **прост**, ако върховете в дъгите му не се повтарят. Цикълът е прост път, в който първият и последният върхове се повтарят.

Цикличен граф е граф G , който съдържа поне един цикъл.

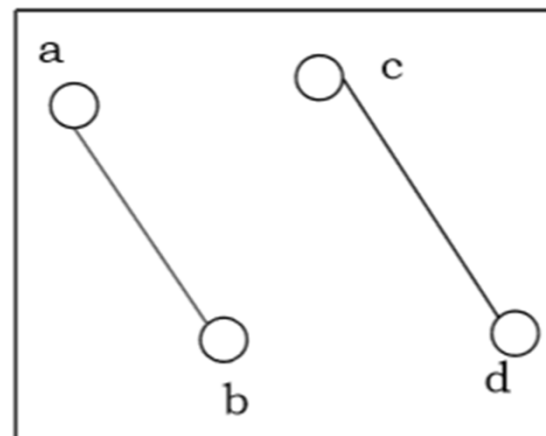
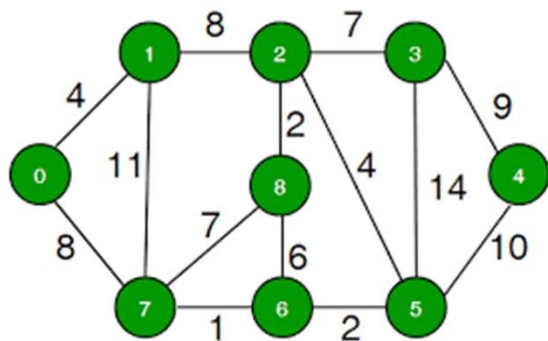


Example of Cyclic Graph

Тема 14 – Мрежови структури

Графът G се нарича **тегловен**, ако дъгите му имат стойности.

Графът G се нарича **свързан**, ако за произволна двойка върхове V_i и V_j съществува път, който ги свързва. В противен случай графът е **несвързан**. В един несвързан граф може да има свързани компоненти, т.е. подграфи, които са свързани.



Тема 14 – Мрежови структури

Съществуват различни начини за представянето на структурата граф в паметта. От статичните могат да се споменат представянето чрез матрица на съседство, чрез матрица на инцидентност, чрез вектори на съседство. Начините са ефективни, когато представяният граф има голям брой дъги и резултантните матрици не са разреждени.

Графът може да се представи и динамично.

Примерна динамична дефиниция на граф

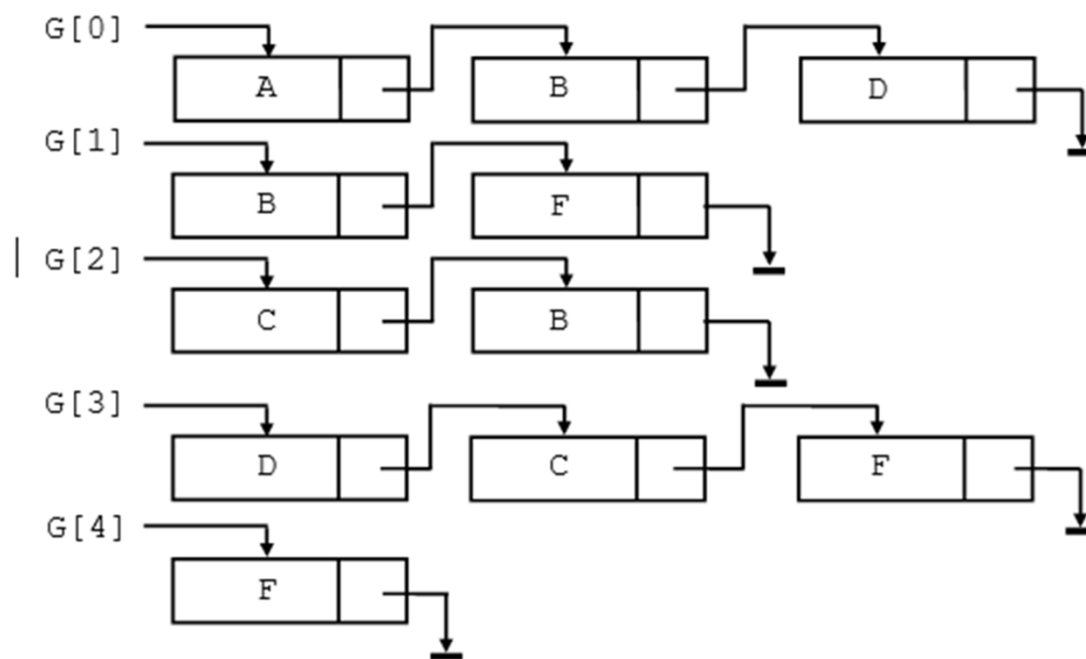
```
struct nd {int key; nd*p1; arc*p2;} //описание на връх  
//p1 - указател към следващия връх  
//p2 - указател към списъка от наследници -  
//всички възли, до които има дъга от даден възел
```

```
struct arc {arc*p3; nd*p4;} //описание на дъга  
//p3 - указател към следващата дъга за даден възел  
//p4 - указател към възел, крайния за дъгата
```

Тема 14 – Мрежови структури

Представяне на граф чрез «списъци на съседство».

Структурата се представя като масив от указатели към списъци, описващи върховете и инцидентните им дъги в графа. Така насоченият граф, даден по-горе, се представя като:



Тема 14 – Мрежови структури

Примерна комбинирана дефиниция на граф

```
const n=10;  
struct link {char key; link *next;} *gr[n];
```

Основните операции със структурата граф са:

1. Инициализиране на граф
2. Търсене на връх
3. Търсене на дъга
4. Включване на връх
5. Включване на дъга
6. Премахване на връх
7. Премахване на дъга

Тема 14 – Мрежови структури

Изброените операции са дефинирани при следните аксиоми:

1. Всеки нов добавян връх е изолиран връх
 2. Ако се включва дъга и върховете, свързани с нея, не принадлежат на графа, те се добавят
 3. Ако се премахва дъга, то върховете, свързани с нея, не се премахват
 4. Ако се премахва връх, то се премахват всички дъги, инцидентни с нея
 5. Включването на нов връх V_i в граф G е възможно, ако V_i не принадлежи на G .
 6. Включването на нова дъга (V_i, V_j) в граф G е възможно, ако тя не принадлежи на G .
 7. Премахването на връх V_i от графа G е възможно, ако V_i принадлежи на G .
 8. Премахването на дъга (V_i, V_j) от графа G е възможно, ако (V_i, V_j) принадлежи на G .
- Ако графът G е неориентиран, трябва да се добави още една аксиома:
9. Ако дъга (V_i, V_j) принадлежи на графа G , то дъгата (V_j, V_i) също така е дъга на графа G .